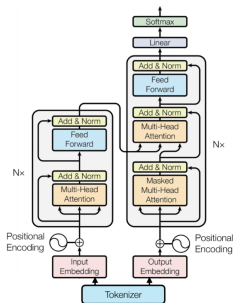


Lecture05: Recurrent Neural Networks, LSTMs, and Transformers



ChatGPT



GPT-4

Please make a syllabus for a course titled "TinyML and Efficient Deep Learning Computing".

Course Syllabus: "TinyML and Efficient Deep Learning Computing"

Course Overview:

This course provides an in-depth exploration of the principles and practices of designing, developing, and deploying efficient deep learning models suitable for resource-constrained environments, with a special emphasis on TinyML (Tiny Machine Learning) applications for edge devices.

Course Objectives:

By the end of this course, students will be able to:

1. Understand the challenges and limitations of deploying deep learning models on edge and embedded devices.
2. Implement model quantization, pruning, and knowledge distillation techniques to optimize model size and computational efficiency.
3. Design, train, and deploy TinyML models tailored for specific edge computing applications.
4. Evaluate trade-offs between model accuracy, size, power consumption, and latency.

Prerequisites:

- * Fundamental understanding of deep learning and neural networks.
- * Basic programming knowledge in Python.
- * Familiarity with TensorFlow or PyTorch.

Week-by-Week Breakdown:

Week 1: Introduction to TinyML and Edge Computing

- * The landscape of edge computing
- * Introduction to TinyML
- * Challenges and opportunities in edge AI

Week 2: Efficient Neural Network Architectures

- * MobileNets, EfficientNet, and other compact architectures
- * Design principles for resource-efficient networks

Week 3: Model Compression Techniques I

- * Quantization: Introduction, types, and benefits
- * Post-training quantization vs. quantization-aware training

Week 4: Model Compression Techniques II

- * Model pruning: Weight pruning, channel pruning, structured pruning
- * Knowledge distillation: Basics and techniques

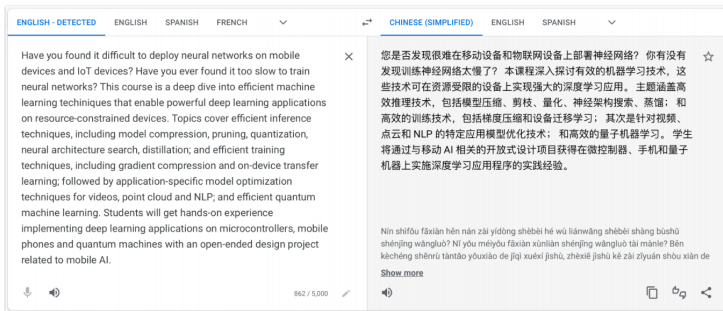
Week 5: Hardware Considerations for TinyML

- * Overview of edge AI hardware
- * Hardware accelerators and NPUs
- * Understanding latency, throughput, and power trade-offs

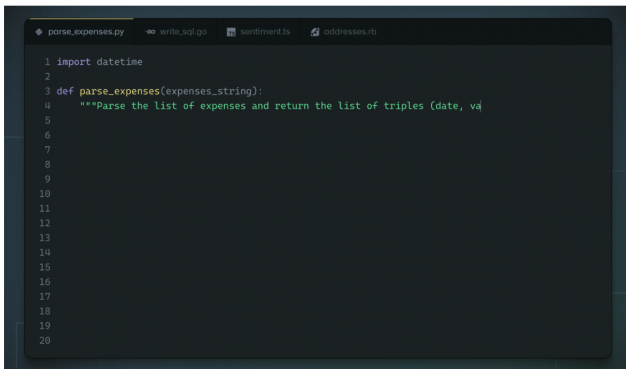
Week 6: TinyML Frameworks and Tools

- * Introduction to TensorFlow Lite (TFLite)

Google Translate

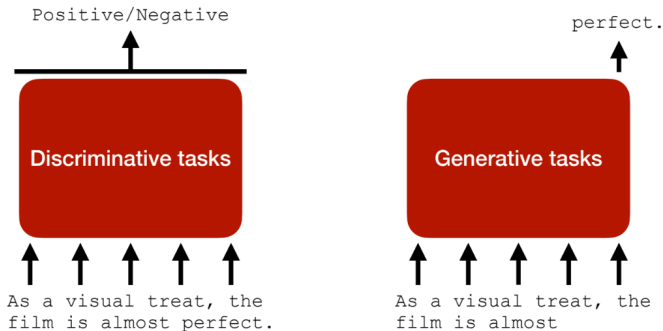


GitHub Copilot

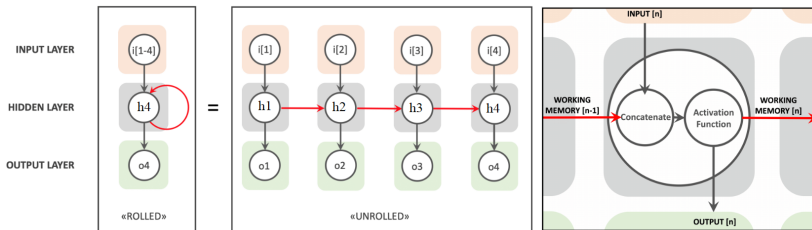


```
parse_expenses.py write_sql.go sentiment.ts addresses.rb
1 import datetime
2
3 def parse_expenses(expenses_string):
4     """Parse the list of expenses and return the list of triples (date, va
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
```


- ▶ **Discriminative tasks**(e.g., sentiment analysis, text classification, textual Entailment)
- ▶ **Generative tasks** (e.g., language modeling, machine translation, summarization)



Recurrent Neural Networks (RNNs)



- We can process a sequence of vectors i by applying a recurrence formula at every time step:

$$h_t = f_W(h_{t-1}, i_t)$$

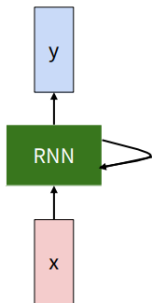
new state old state input vector at some time step
 some function with parameters W

$$o_t = f_{W_{hy}}(h_t)$$

output another function with parameters W_{hy} new state

Notice: the same function and the same set of parameters are used at every time step.

The state consists of a single “hidden” vector h :



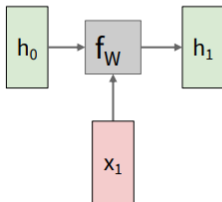
$$h_t = f_W(h_{t-1}, x_t)$$

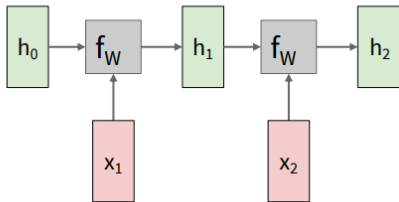


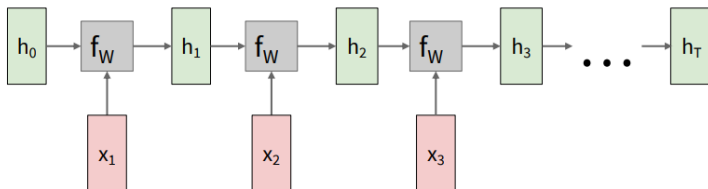
$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$

$$y_t = W_{hy}h_t$$

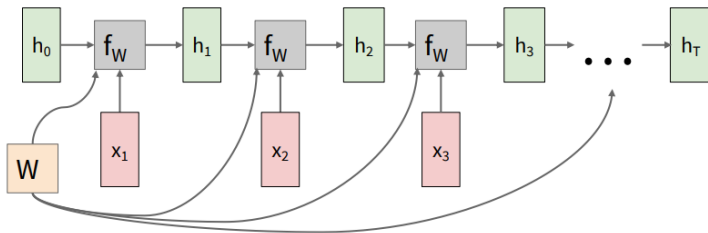
Sometimes called a “Vanilla RNN” or an “Elman RNN” after Prof. Jeffrey Elman



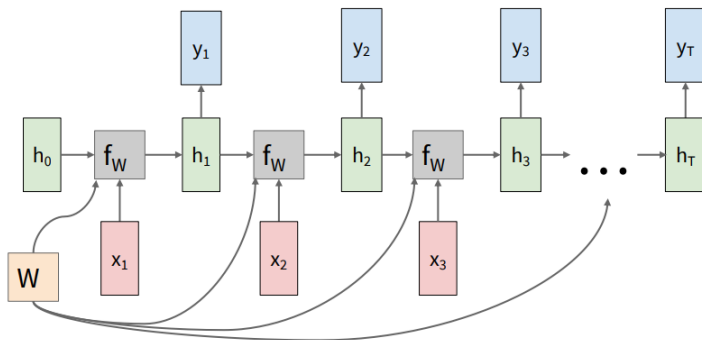




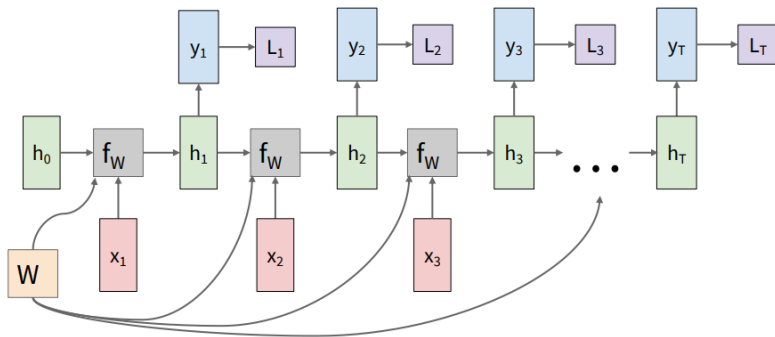
Re-use the same weight matrix at every time-step



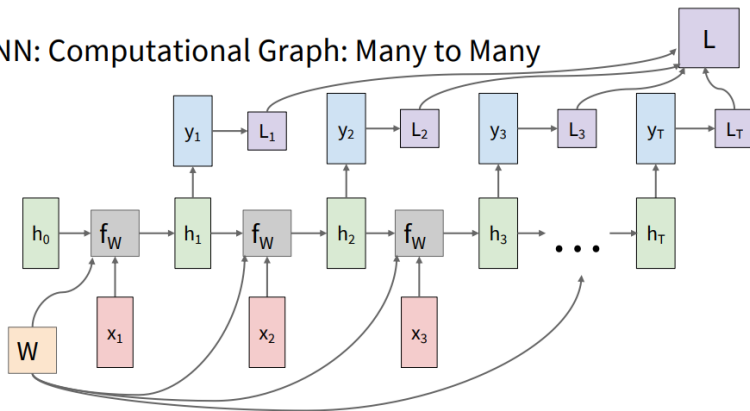
RNN: Computational Graph: Many to Many



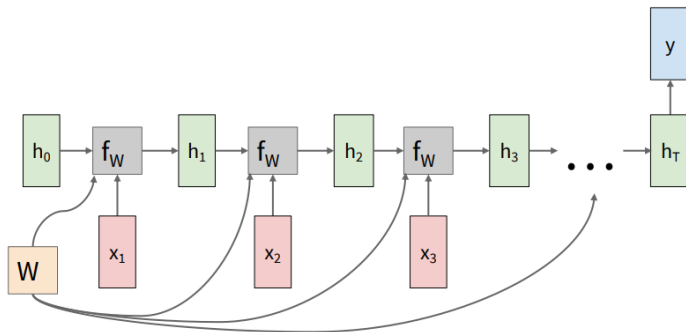
RNN: Computational Graph: Many to Many



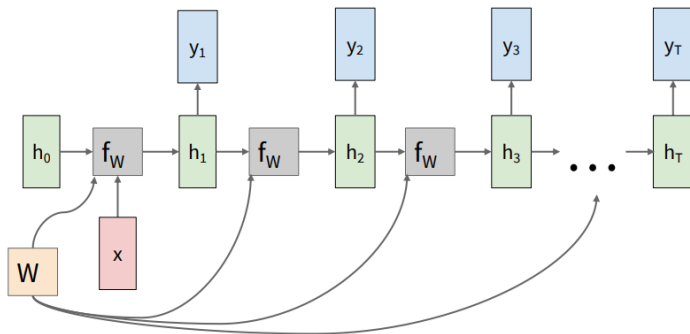
RNN: Computational Graph: Many to Many



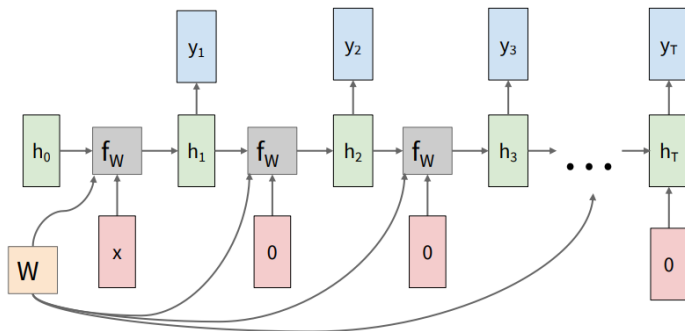
RNN: Computational Graph: Many to One



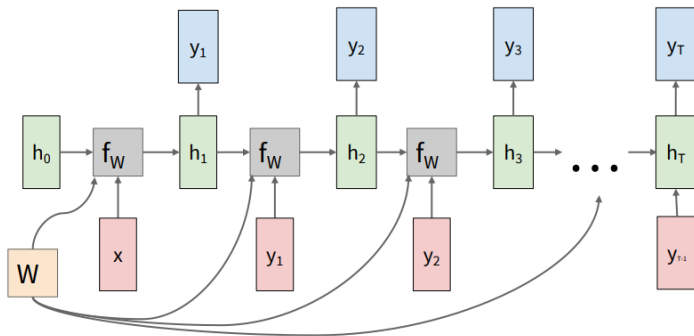
RNN: Computational Graph: One to Many



RNN: Computational Graph: One to Many



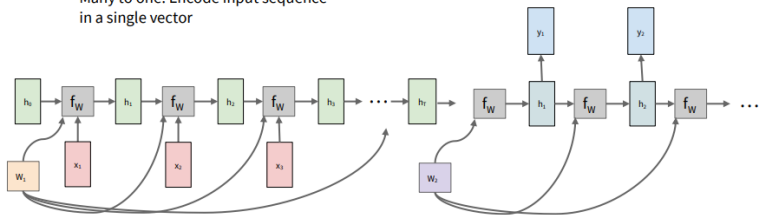
RNN: Computational Graph: One to Many



Sequence to Sequence: Many-to-one + one-to-many

Many to one: Encode input sequence in a single vector

One to many: Produce output sequence from single input vector

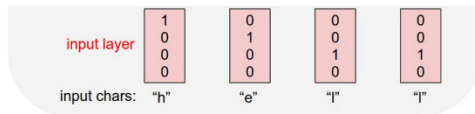


Sutskever et al, "Sequence to Sequence Learning with Neural Networks", NIPS 2014

Example:
Character-level
Language Model

Vocabulary:
[h,e,l,o]

Example training
sequence:
"hello"

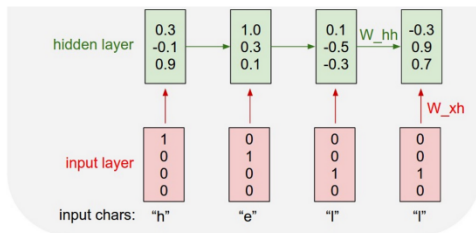


Example:
Character-level
Language Model

Vocabulary:
[h,e,l,o]

Example training
sequence:
“hello”

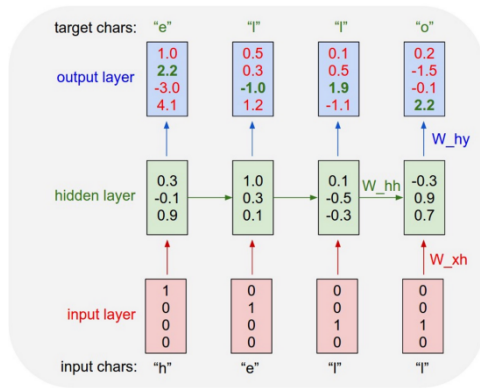
$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$



Example:
Character-level
Language Model

Vocabulary:
[h,e,l,o]

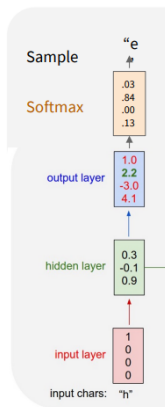
Example training
sequence:
“hello”



Example: Character-level Language Model **Sampling**

Vocabulary:
[h,e,l,o]

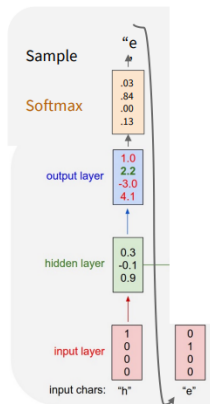
At test-time sample characters
one at a time, feed back to
model



Example: Character-level Language Model Sampling

Vocabulary:
[h,e,l,o]

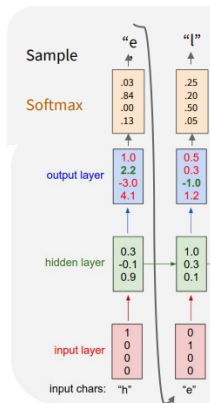
At test-time sample characters
one at a time, feed back to
model



Example: Character-level Language Model Sampling

Vocabulary:
[h,e,l,o]

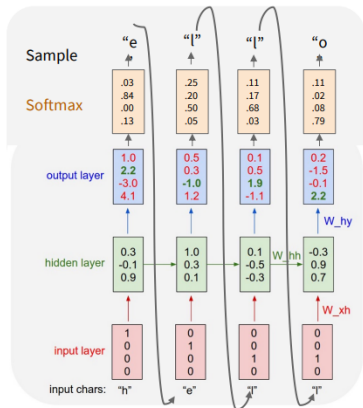
At test-time sample characters
one at a time, feed back to
model

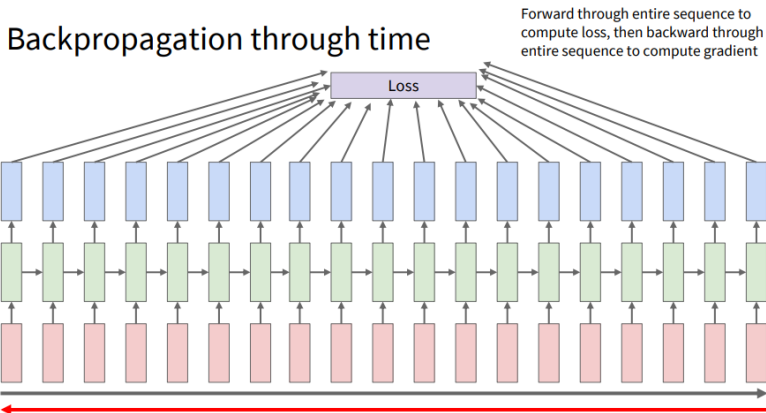


Example: Character-level Language Model Sampling

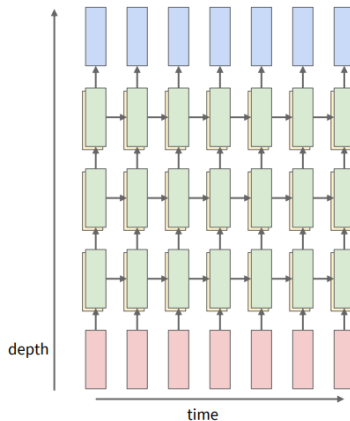
Vocabulary:
[h,e,l,o]

At test-time sample characters
one at a time, feed back to
model



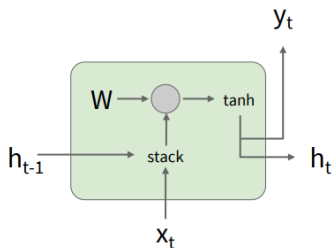


Multilayer RNNs



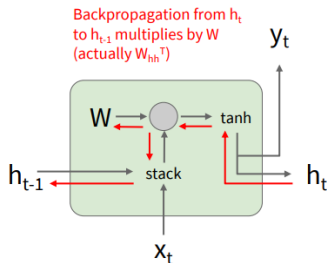
Vanilla RNN Gradient Flow

Bengio et al, "Learning long-term dependencies with gradient descent is difficult", IEEE Transactions on Neural Networks, 1994
Pascanu et al, "On the difficulty of training recurrent neural networks", ICML 2013



$$\begin{aligned} h_t &= \tanh(W_{hh}h_{t-1} + W_{hx}x_t) \\ &= \tanh\left((W_{hh} \quad W_{hx}) \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right) \\ &= \tanh\left(W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right) \end{aligned}$$

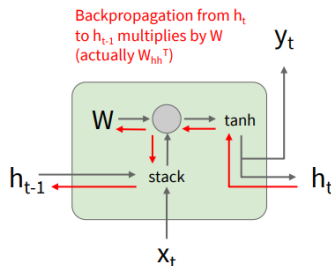
Vanilla RNN Gradient Flow



Bengio et al, "Learning long-term dependencies with gradient descent is difficult", IEEE Transactions on Neural Networks, 1994
Pascanu et al, "On the difficulty of training recurrent neural networks", ICML 2013

$$\begin{aligned}h_t &= \tanh(W_{hh}h_{t-1} + W_{hx}x_t) \\&= \tanh\left((W_{hh} \quad W_{hx}) \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right) \\&= \tanh\left(W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right)\end{aligned}$$

Vanilla RNN Gradient Flow



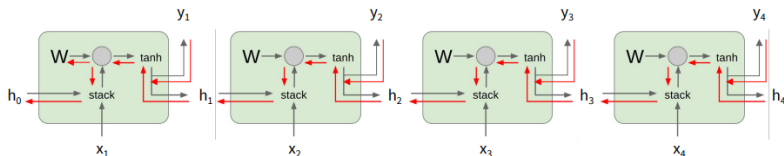
Bengio et al, "Learning long-term dependencies with gradient descent is difficult", IEEE Transactions on Neural Networks, 1994
Pascanu et al, "On the difficulty of training recurrent neural networks", ICML 2013

$$\begin{aligned} h_t &= \tanh(W_{hh}h_{t-1} + W_{hx}x_t) \\ &= \tanh\left((W_{hh} \quad W_{hx}) \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right) \\ &= \tanh\left(W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right) \end{aligned}$$

$$\frac{\partial h_t}{\partial h_{t-1}} = \tanh'(W_{hh}h_{t-1} + W_{hx}x_t)W_{hh}$$

Vanilla RNN Gradient Flow

Bengio et al, "Learning long-term dependencies with gradient descent is difficult", IEEE Transactions on Neural Networks, 1994
Pascanu et al, "On the difficulty of training recurrent neural networks", ICML 2013

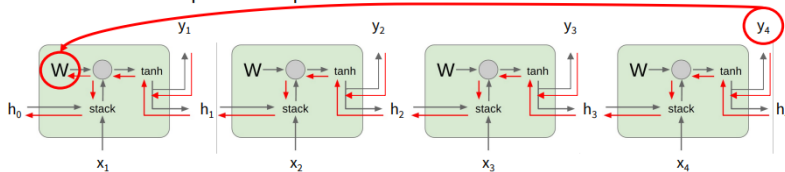


$$\frac{\partial L}{\partial W} = \sum_{t=1}^T \frac{\partial L_t}{\partial W}$$

Vanilla RNN Gradient Flow

Gradients over multiple time steps:

Bengio et al, "Learning long-term dependencies with gradient descent is difficult", IEEE Transactions on Neural Networks, 1994
Pascanu et al, "On the difficulty of training recurrent neural networks", ICML 2013



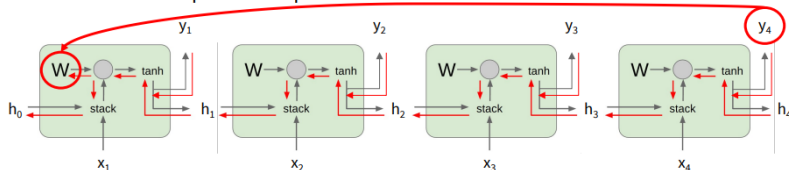
$$\frac{\partial L}{\partial W} = \sum_{t=1}^T \frac{\partial L_t}{\partial W}$$

$$\frac{\partial L_T}{\partial W} = \frac{\partial L_T}{\partial h_T} \frac{\partial h_t}{\partial h_{t-1}} \cdots \frac{\partial h_1}{\partial W}$$

Vanilla RNN Gradient Flow

Gradients over multiple time steps:

Bengio et al, "Learning long-term dependencies with gradient descent is difficult", IEEE Transactions on Neural Networks, 1994
Pascanu et al, "On the difficulty of training recurrent neural networks", ICML 2013



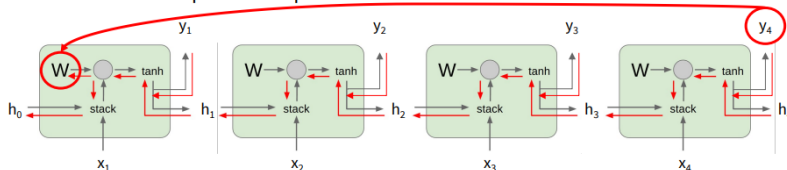
$$\frac{\partial L}{\partial W} = \sum_{t=1}^T \frac{\partial L_t}{\partial W}$$

$$\frac{\partial L_T}{\partial W} = \frac{\partial L_T}{\partial h_T} \frac{\partial h_T}{\partial h_{T-1}} \cdots \frac{\partial h_1}{\partial W} = \frac{\partial L_T}{\partial h_T} \left(\prod_{t=2}^T \frac{\partial h_t}{\partial h_{t-1}} \right) \frac{\partial h_1}{\partial W}$$

Vanilla RNN Gradient Flow

Gradients over multiple time steps:

Bengio et al, "Learning long-term dependencies with gradient descent is difficult", IEEE Transactions on Neural Networks, 1994
Pascanu et al, "On the difficulty of training recurrent neural networks", ICML 2013



$$\frac{\partial L}{\partial W} = \sum_{t=1}^T \frac{\partial L_t}{\partial W}$$

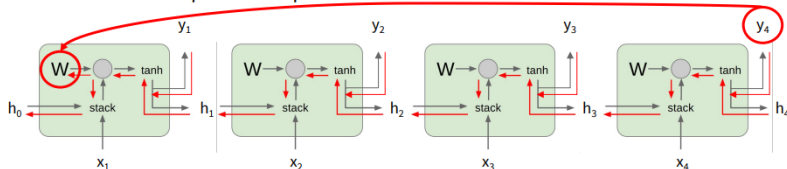
$$\frac{\partial h_t}{\partial h_{t-1}} = \tanh'(W_{hh}h_{t-1} + W_{hx}x_t)W_{hh}$$

$$\frac{\partial L_T}{\partial W} = \frac{\partial L_T}{\partial h_T} \frac{\partial h_t}{\partial h_{t-1}} \cdots \frac{\partial h_1}{\partial W} = \frac{\partial L_T}{\partial h_T} \left(\prod_{t=2}^T \frac{\partial h_t}{\partial h_{t-1}} \right) \frac{\partial h_1}{\partial W}$$

Vanilla RNN Gradient Flow

Gradients over multiple time steps:

Bengio et al, "Learning long-term dependencies with gradient descent is difficult", IEEE Transactions on Neural Networks, 1994
Pascanu et al, "On the difficulty of training recurrent neural networks", ICML 2013



$$\frac{\partial L}{\partial W} = \sum_{t=1}^T \frac{\partial L_t}{\partial W}$$

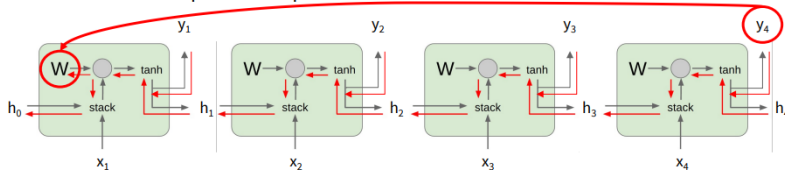
Almost always < 1
Vanishing gradients

$$\frac{\partial L_T}{\partial W} = \frac{\partial L_T}{\partial h_T} \left(\prod_{t=2}^T \boxed{\tanh'(W_{hh} h_{t-1} + W_{xh} x_t)} \right) W_{hh}^{T-1} \frac{\partial h_1}{\partial W}$$

Vanilla RNN Gradient Flow

Gradients over multiple time steps:

Bengio et al, "Learning long-term dependencies with gradient descent is difficult", IEEE Transactions on Neural Networks, 1994
Pascanu et al, "On the difficulty of training recurrent neural networks", ICML 2013



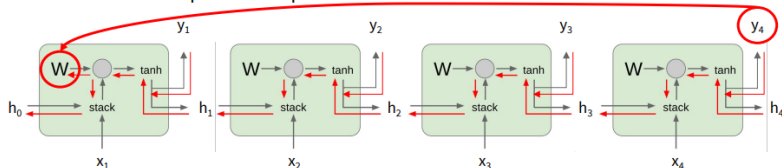
$$\frac{\partial L}{\partial W} = \sum_{t=1}^T \frac{\partial L_t}{\partial W}$$

What if we assumed no non-linearity?

Vanilla RNN Gradient Flow

Gradients over multiple time steps:

Bengio et al, "Learning long-term dependencies with gradient descent is difficult", IEEE Transactions on Neural Networks, 1994
Pascanu et al, "On the difficulty of training recurrent neural networks", ICML 2013



What if we assumed no non-linearity?

$$\frac{\partial L}{\partial W} = \sum_{t=1}^T \frac{\partial L_t}{\partial W}$$

Largest singular value > 1:
Exploding gradients

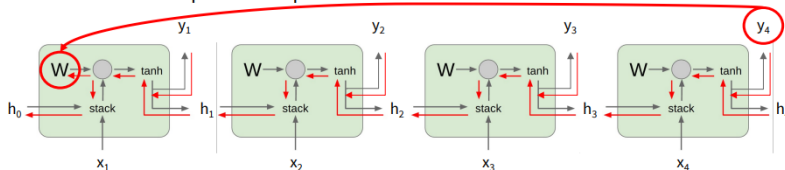
$$\frac{\partial L_T}{\partial W} = \frac{\partial L_T}{\partial h_T} \boxed{W_{hh}^{T-1}} \frac{\partial h_1}{\partial W}$$

Largest singular value < 1:
Vanishing gradients

Vanilla RNN Gradient Flow

Gradients over multiple time steps:

Bengio et al, "Learning long-term dependencies with gradient descent is difficult", IEEE Transactions on Neural Networks, 1994
Pascanu et al, "On the difficulty of training recurrent neural networks", ICML 2013



What if we assumed no non-linearity?

$$\frac{\partial L}{\partial W} = \sum_{t=1}^T \frac{\partial L_t}{\partial W}$$

Largest singular value > 1:
Exploding gradients

$$\frac{\partial L_T}{\partial W} = \frac{\partial L_T}{\partial h_T} \boxed{W_{hh}^{T-1}} \frac{\partial h_1}{\partial W}$$

Largest singular value < 1:
Vanishing gradients

→ Change RNN architecture

Long Short Term Memory (LSTM)

Vanilla RNN

$$h_t = \tanh \left(W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix} \right)$$

LSTM

Four gates

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

Cell state

$$c_t = f \odot c_{t-1} + i \odot g$$

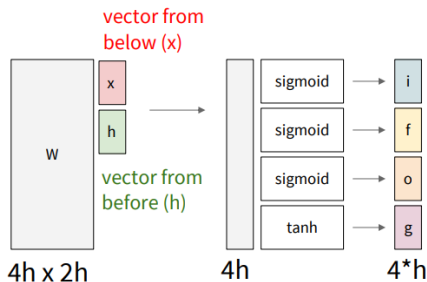
Hidden state

$$h_t = o \odot \tanh(c_t)$$

Hochreiter and Schmidhuber, "Long Short Term Memory", Neural Computation 1997

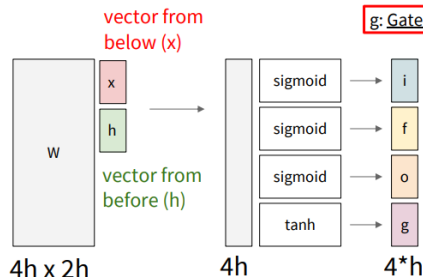
Long Short Term Memory (LSTM)

[Hochreiter et al., 1997]



Long Short Term Memory (LSTM)

[Hochreiter et al., 1997]



$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

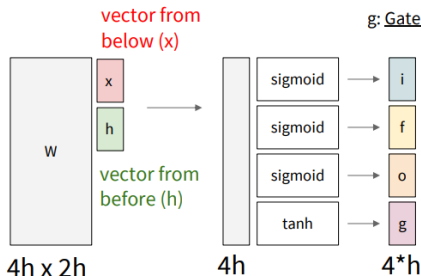
$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

Long Short Term Memory (LSTM)

[Hochreiter et al., 1997]

i: Input gate, whether to write to cell



g: Gate gate (?), How much to write to cell

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$
$$h_t = o \odot \tanh(c_t)$$

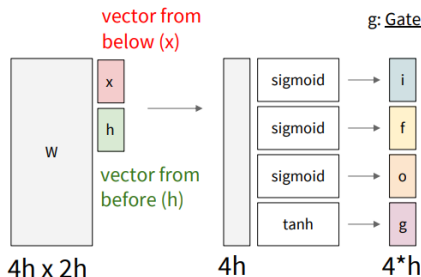
Long Short Term Memory (LSTM)

[Hochreiter et al., 1997]

i: Input gate, whether to write to cell

f: Forget gate, Whether to erase cell

g: Gate gate (?), How much to write to cell



$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

Long Short Term Memory (LSTM)

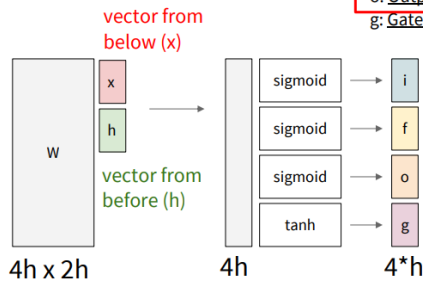
[Hochreiter et al., 1997]

i: Input gate, whether to write to cell

f: Forget gate, Whether to erase cell

o: Output gate, How much to reveal cell

g: Gate gate (?), How much to write to cell



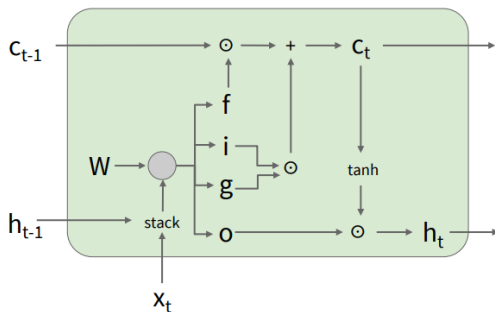
$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

Long Short Term Memory (LSTM)

[Hochreiter et al., 1997]



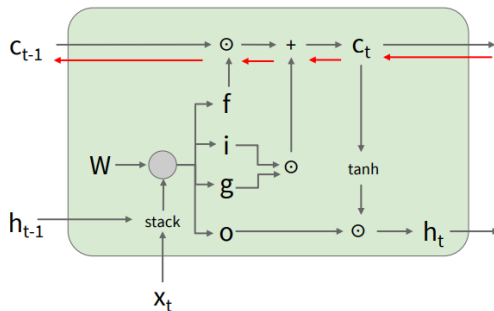
$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

Long Short Term Memory (LSTM): Gradient Flow

[Hochreiter et al., 1997]



Backpropagation from c_t to c_{t-1} only elementwise multiplication by f , no matrix multiply by W

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

Do LSTMs solve the vanishing gradient problem?

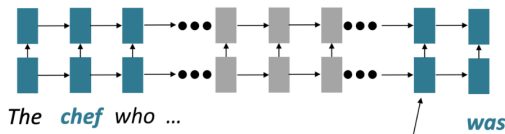
The LSTM architecture makes it easier for the RNN to preserve information over many timesteps

- e.g. if the $f = 1$ and the $i = 0$, then the information of that cell is preserved indefinitely.
- By contrast, it's harder for vanilla RNN to learn a recurrent weight matrix W_h that preserves info in hidden state

LSTM doesn't guarantee that there is no vanishing/exploding gradient, but it does provide an easier way for the model to learn long-distance dependencies

► Hard to model long-term relationships

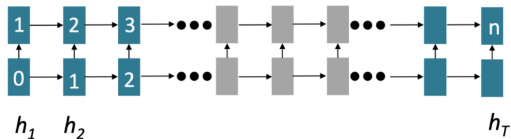
1. It takes $O(\text{seq-len})$ steps to model the interaction between two tokens
2. Images have locality; but languages may not have locality:



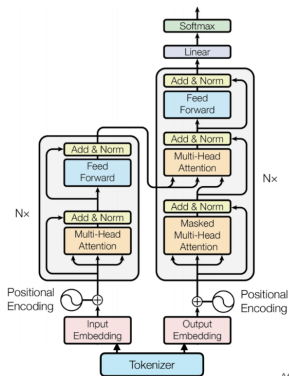
Info of *chef* has gone through $O(\text{sequence length})$ many layers!

► Limited training parallelism

1. The states are strictly dependent on earlier states
2. It takes n steps to get state n



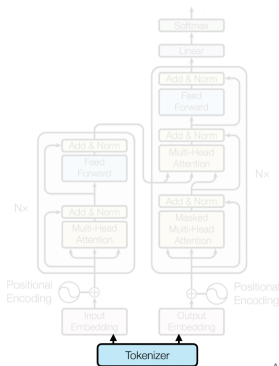
Numbers indicate min # of steps before a state can be computed



- Tokenize words (word $\rightarrow$ tokens)
- Map tokens into embeddings
- Embeddings go through Transformer blocks
 - Multi-Head Attention (MHA)
 - Feed-Forward Network (FFN)
 - LayerNorm
 - Residual connection
- Positional encoding
- Final prediction with Linear head

Attention Is All You Need [Vaswani *et al.*, 2017]

Transformer



- Tokenize words (word $\rightarrow$ tokens)
- Map tokens into embeddings
- Embeddings go through transformer blocks
 - Multi-Head Attention (MHA)
 - Feed-Forward Network (FFN)
 - LayerNorm
 - Residual connection
- Positional encoding
- Final prediction with linear head

Attention Is All You Need [Vaswani et al., 2017]

Tokenization

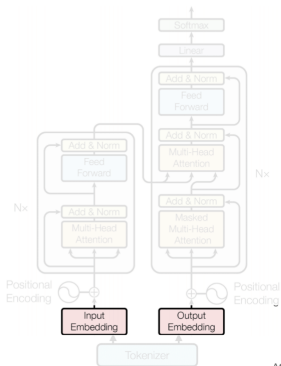
- A tokenizer maps a word to one/multiple tokens:
- **110 words => 162 tokens**

Large generative models (e.g., large language models, diffusion models) have shown remarkable performance, but they require a massive amount of computational resources. To make them more accessible, it is crucial to improve their efficiency. This course will introduce efficient AI computing techniques that enable powerful deep learning applications on resource-constrained devices. Topics include model compression, pruning, quantization, neural architecture search, distributed training, data/model parallelism, gradient compression, and on-device fine-tuning. It also introduces application-specific acceleration techniques for large language models, diffusion models, video recognition, and point cloud. This course will also cover topics about quantum machine learning. Students will get hands-on experience deploying large language models (e.g., LLaMA 2) on a laptop.

TEXT	TOKEN IDS
Large generative models (e.g., large language models, diffusion models)	[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100, 101, 102, 103, 104, 105, 106, 107, 108, 109, 110, 111, 112, 113, 114, 115, 116, 117, 118, 119, 120, 121, 122, 123, 124, 125, 126, 127, 128, 129, 130, 131, 132, 133, 134, 135, 136, 137, 138, 139, 140, 141, 142, 143, 144, 145, 146, 147, 148, 149, 150, 151, 152, 153, 154, 155, 156, 157, 158, 159, 160, 161, 162]

<https://platform.openai.com/tokenizer>

Transformer



Attention Is All You Need [Vaswani *et al.*, 2017]

- Tokenize words (word -> tokens)
- **Map tokens into embeddings**
- Embeddings go through transformer blocks
 - Multi-Head Attention (MHA)
 - Feed-Forward Network (FFN)
 - LayerNorm
 - Residual connection
- Positional encoding
- Final prediction with linear head

Word Representation

One-Hot Encoding

- **Key idea:** Representing each word as a vector that has as many values in it as there are words in the vocabulary. Each column in a vector represents one possible word in a vocabulary.

~100k columns, only one 1 in each vector

the →	1	0	0	0	...	0	0	0
good →	0	0	1	0	...	0	0	0
movie →	0	0	0	0	...	1	0	0

- For **large vocabularies**, these vectors can get very **long**, and they contain all 0's except for one value. This is considered a very sparse representation.

Content credit: https://cezannec.github.io/CNN_Text_Classification/

Word Representation

Word Embedding

- **Key idea:** Map the word index to a **continuous** word embedding through a **look-up table**.

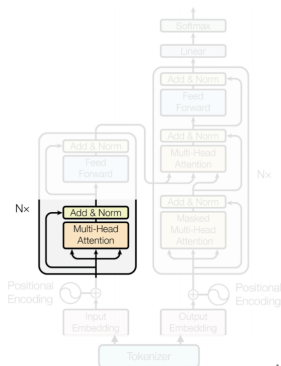
~300 columns

the →	0.2	0.4	-0.1
good →	0.7	-0.5	0.3
movie →	0.1	0.2	0.6

- The word embedding can be trained **end-to-end** with the model for the downstream tasks.
 - Popular pre-trained word embeddings: Word2Vec, GloVe.

Content credit: https://cezannec.github.io/CNN_Text_Classification/

Transformer



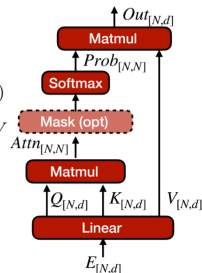
- Tokenize words (word $\rightarrow$ tokens)
- Map tokens into embeddings
- Embeddings go through transformer blocks
 - **Multi-Head Attention (MHA)**
 - **Feed-Forward Network (FFN)**
 - **LayerNorm**
 - **Residual connection**
- **Positional encoding**
- **Final prediction with linear head**

Attention Is All You Need (Vaswani et al. 2017)

Self-Attention

- Project the embedding E into query, key, and value (Q, K, V)
- The Query-Key-Value design is analogous to a retrieval system (let's take YouTube search as an example)
 - Query: text prompt in the search bar
 - Key: the titles/descriptions of videos
 - Value: the corresponding videos
- Multiply Q and K to get the inner product (normalize by $\sqrt{d}$)
- Pass through Softmax to get the attention weights of shape $N \times N$
 - The attention computation has $O(N^2)$ complexity!
- Multiply attention weights with V to get the output

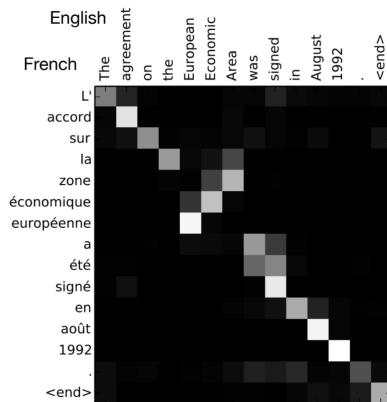
$$\begin{aligned} \text{Attention}(Q, K, V) \\ = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \end{aligned}$$



Content credit: <https://stats.stackexchange.com/questions/421935/what-exactly-are-keys-queries-and-values-in-attention-mechanisms>

Self-Attention

- Example of attention matrix from machine translation
- The attention mechanism finds the correspondence between words



Example from Greg Durrett

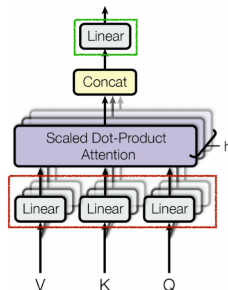
Multi-Head Attention (MHA)

Each head captures different semantics

- We need different attention maps to capture different semantic relationships
- Multi-head attention models different attention functionalities by having $H > 1$ heads (parallel branches of QKV attention)
- The final output is concatenated and goes through a linear projection to merge the features

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

where $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$

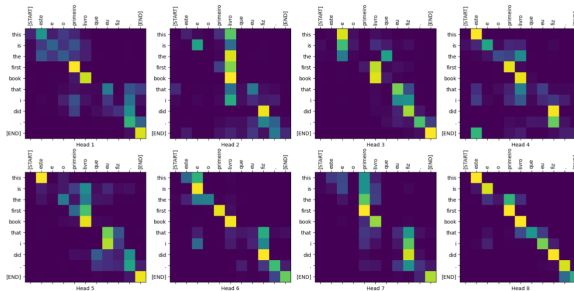


Content credit: https://cezanne.github.io/CNN_Text_Classification/

Multi-Head Attention (MHA)

Each head captures different semantics

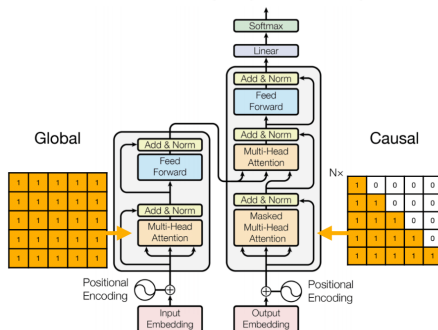
- Visualization from different heads



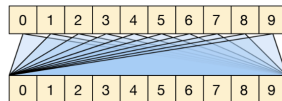
<https://www.tensorflow.org/text/tutorials/transformer>

Attention Masking

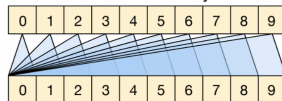
Casual/masked language modeling



- Global self attention layer

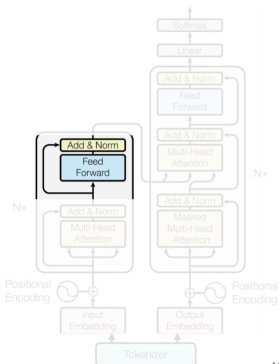


- Causal self attention layer



<https://www.tensorflow.org/text/tutorials/transformer>

Transformer



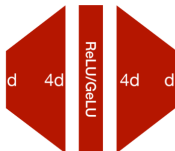
- Tokenize words (word -> tokens)
- Map tokens into embeddings
- Embeddings go through transformer blocks
 - Multi-Head Attention (MHA)
 - **Feed-Forward Network (FFN)**
 - LayerNorm
 - Residual connection
- Positional encoding
- Final prediction with linear head

Attention Is All You Need [Vaswani et al., 2017]

Feed-Forward Network (FFN)

- Self-attention models relationships **between** tokens, but there is no **element-wise** non-linearity
- Add a feed-forward network (FFN) to help with feature modeling
- The vanilla implementation is a two-layer MLP with a larger hidden state size (**inverted bottleneck**) and ReLU/GeLU **activation**

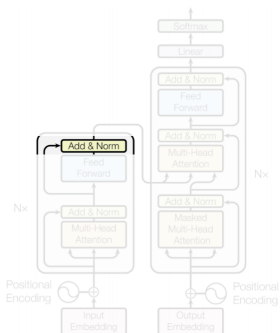
$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$



- Later we are going to visit several design variants

Model	#L	#H	d_{model}	LR	Batch
125M	12	12	768	$6.0e-4$	0.5M
350M	24	16	1024	$3.0e-4$	0.5M
1.3B	24	32	2048	$2.0e-4$	1M
2.7B	32	32	2560	$1.6e-4$	1M
6.7B	32	32	4096	$1.2e-4$	2M
13B	40	40	5120	$1.0e-4$	4M
30B	48	56	7168	$1.0e-4$	4M
66B	64	72	9216	$0.8e-4$	2M
175B	96	96	12288	$1.2e-4$	2M

Transformer

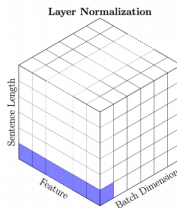
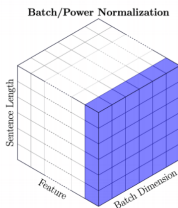


- Tokenize words (word $\rightarrow$ tokens)
- Map tokens into embeddings
- Embeddings go through transformer blocks
 - Multi-Head Attention (MHA)
 - Feed-Forward Network (FFN)
 - LayerNorm
 - Residual connection
- Positional encoding
- Final prediction with linear head

Attention Is All You Need [Vaswani et al., 2017]

Layer Normalization (LN)

- Unlike BatchNorm (BN) in CNNs, Transformers use **LayerNorm (LN)**
- LN performs normalization for each token's embedding, then applies a learnable affine transformation

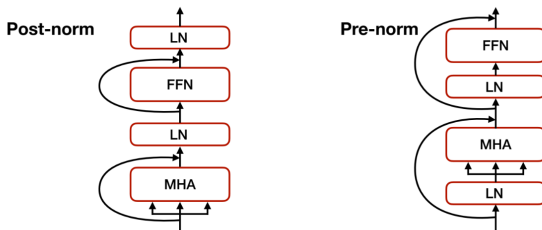


$$y = \frac{x - \mathbb{E}[x]}{\sqrt{\text{Var}[x] + \epsilon}} * \gamma + \beta$$

Image credit: <http://proceedings.mlr.press/v119/shen20e/shen20e.pdf>

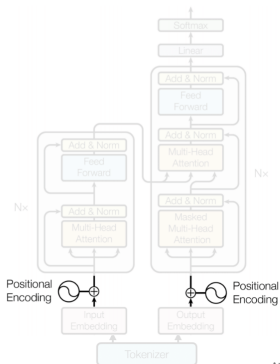
Transformer Block

- Now we put everything together to build a Transformer block
- We add LayerNorm and Residual Connections for training stability
- **Note:** the pre-norm design is more popular now due to better training stability



<https://pytorch.org/docs/stable/generated/torch.nn.LayerNorm.html>

Transformer



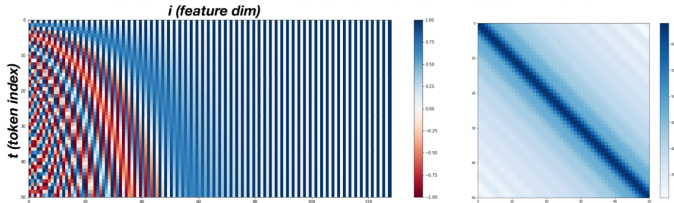
- Tokenize words (word $\rightarrow$ tokens)
- Map tokens into embeddings
- Embeddings go through transformer blocks
 - Multi-Head Attention (MHA)
 - Feed-Forward Network (FFN)
 - LayerNorm
 - Residual connection
- Positional encoding
- Final prediction with linear head
- **Problem:** attention and FFN do not differentiate the **order** of the input tokens (unlike convolutions)
 - Set encoding (bad) instead of sequence encoding (good)
- **Solution: Positional encoding!**

Attention Is All You Need [Vaswani et al., 2017]

Position Encoding (PE)

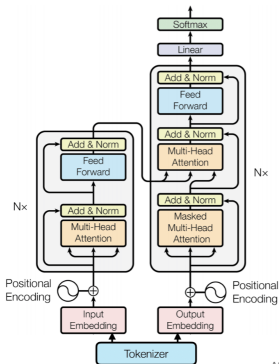
- Positional encoding provides positional information.
 - **Unique** encoding for each word's position in a sentence.
 - Each element is a function of t (i.e., index, absolute PE)
 - Added to the raw word embedding to fuse in positional information

$$\vec{p}_t^{(i)} = f(t)^{(i)} := \begin{cases} \sin(\omega_k \cdot t), & \text{if } i = 2k \\ \cos(\omega_k \cdot t), & \text{if } i = 2k + 1 \end{cases} \quad \omega_k = \frac{1}{10000^{2k/d}}$$



Content credit: https://kazemnejad.com/blog/transformer_architecture_positional_encoding/

Putting it together



- Tokenize words (word $\rightarrow$ tokens)
- Map tokens into embeddings
- Embeddings go through transformer blocks
 - Multi-Head Attention (MHA)
 - Feed-Forward Network (FFN)
 - LayerNorm
 - Residual connection
- Positional encoding
- Final prediction with linear head

Attention Is All You Need [Vaswani et al., 2017]

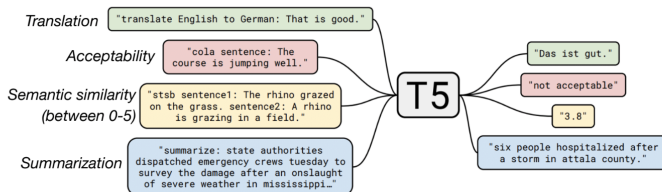
Transformer Design Variants

Improved designs after the initial transformer paper

- Most designs from the initial Transformer paper have been widely used by the community
- Nonetheless, people proposed various alternative designs. For example:
 - Encoder-decoder (T5), encoder-only (BERT), decoder-only (GPT)

Encoder-Decoder

- The original Transformer is an Encoder-Decoder architecture
- T5 offers a unified text-to-text model for transfer learning on various NLP tasks



Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer [Raffel et al., 2019]

Encoder-Decoder

- The original Transformer is an Encoder-Decoder architecture
- T5 offers a unified text-to-text model for transfer learning on various NLP tasks
- The **prompt** is fed to the encoder, the decoder generates the **answer**.

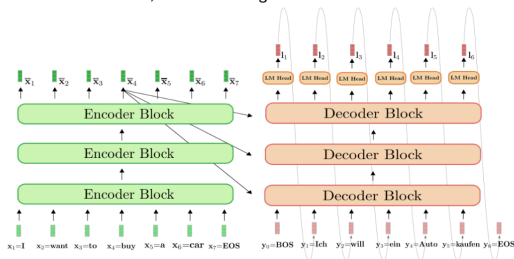


Image credit: <https://huggingface.co/blog/encoder-decoder>

Encoder-only (BERT)

Bidirectional Encoder Representations from Transformers (BERT)

- BERT is an **encoder-only** language model
- Two pre-training objectives
 - Task #1: **Masked Language Model (MLM)**
 - Mask some percentage (15%) of the input tokens at random; train model to predict masked tokens
 - Task #2: **Next Sentence Prediction (NSP)**
 - Whether sentence B is the next sentence of sentence A (less used)
- The pre-trained model is fine-tuned to downstream tasks

Attention mask:

1	1	1	1	1
1	1	1	1	1
1	1	1	1	1
1	1	1	1	1
1	1	1	1	1

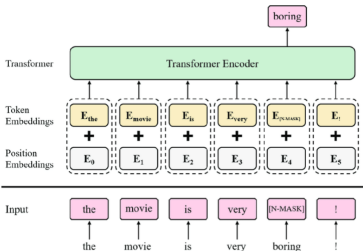


Image credit: <https://www.mdpi.com/2073-8994/11/11/1393/htm>

BERT: Pre-Training of Deep Bidirectional Transformers for Language Understanding [Devlin et al., 2018]

Decoder-only (GPT)

Generative Pre-trained Transformer (GPT)

- GPT is a **decoder-only** language model
- Pre-training objective:
 - Next word prediction

$$L_1(\mathcal{U}) = \sum_i \log P(u_i | u_{i-k}, \dots, u_{i-1}; \Theta)$$

- For smaller models (GPT-2), the pre-trained model is fine-tuned to downstream tasks
- Larger model can run in zero-shot/few-shot

Attention mask:

1	0	0	0	0
1	1	0	0	0
1	1	1	0	0
1	1	1	1	0
1	1	1	1	1

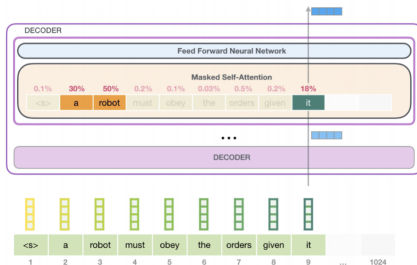


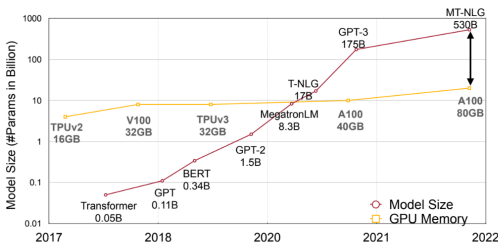
Image credit: <https://jalammar.github.io/illustrated-gpt2/>

Improving Language Understanding by Generative Pre-Training [Radford *et al.*, 2018]

Large Language Models (LLMs)

The growing trend of scaling up

- LLMs are scaled-up transformers models trained on large language corpus (natural language, code, etc.)

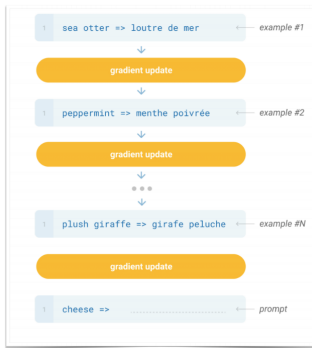


SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models [Xiao et al., 2022]

GPT-3

Scaling up Transformers to be few-shot learners (in-context learning)

- Traditional NLP pipeline:
 - Pretraining
 - Fine-tuning** on downstream tasks



Language Models are Few-Shot Learners [Brown *et al.*, 2020]

GPT-3

Scaling up Transformers to be few-shot learners (in-context learning)

- Scaled-up LLM (175B) can generalize to new tasks **w/o** fine-tuning by either
 - **Zero-shot**: answer questions given task descriptions
 - **Few-shot**: learn with demonstrations

Zero-shot

The model predicts the answer given only a natural language description of the task. No gradient updates are performed.

```
1 Translate English to French: ← task description
2 cheese => ..... ← prompt
```

Few-shot

In addition to the task description, the model sees a few examples of the task. No gradient updates are performed.

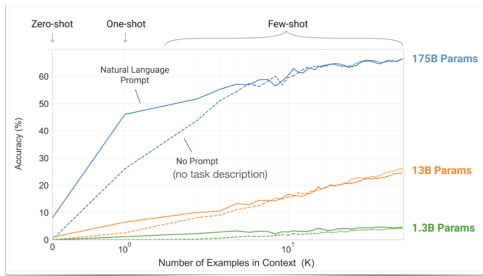
```
1 Translate English to French: ← task description
2 sea otter => loutre de mer ← examples
3 peppermint => menthe poivrée ←
4 plush girafe => girafe peluche ←
5 cheese => ..... ← prompt
```

Language Models are Few-Shot Learners [Brown *et al.*, 2020]

GPT-3

Scaling up Transformers to be few-shot learners (in-context learning)

- Scaled-up LLM (175B) can generalize to new tasks **w/o** fine-tuning by either
 - **Larger** models can more **effectively** utilize few-shot demonstrations
 - With more demonstrations, the no-prompt accuracy catches up with the task-prompted accuracy

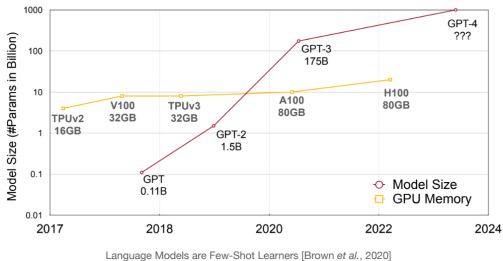


Language Models are Few-Shot Learners [Brown et al., 2020]

GPT Family

The history of scaling for GPT models

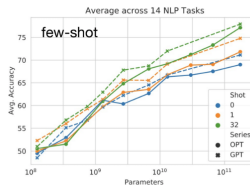
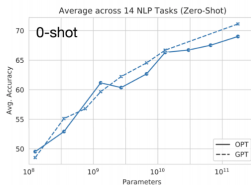
- GPT-1 (2018): 117 million
- GPT-2 (2019): 1.5B
- GPT-3 (2020): 175B
- GPT-4 (2023): ???



OPT

Open Pre-trained Transformer Language Models

- Open-source pre-trained LLM from Meta
- Size: 125M/350M/1.3B/2.7B/6.7B/13B/30B/66B/175B
- Design choices: Decoder-only, Pre-norm (post-norm for 350M), ReLU activation in FFN
- 175B model: hidden dimension 12288, #heads 96, vocab size 50k, context length 2048
- Close performance compared to GPT models



OPT: Open Pre-trained Transformer Language Models [Zhang *et al.*, 2022]

BLOOM

Open Pre-trained Transformer Language Models

- Open-source pre-trained LLM from BigScience (and opensource community)
- Size: 560M/1.1B/1.7B/3B/7.1B/176B
- Design choices: Decoder-only, Pre-norm, GeLU activation in FFN, ALiBi positional embedding
- 176B model: hidden dimension 14336, #heads 112, vocab size 250k, context length 2048
- Support multi-lingual (59 languages in the corpus)



BLOOM: A 176B-Parameter Open-Access Multilingual Language Model [BigScience, 2022]

LLaMA

Significantly better opensource LLMs

- Open-source pre-trained LLM from Meta
- Size: 7B/13B/33B/65B
- Design choices: Decoder-only, Pre-norm, **SwiGLU (swish, gated linear units)**, rotary positional embedding (RoPE)
- 7B model: hidden dimension 4096, #heads 32, #layers 32, vocab size 32k, context length 2048
- 65B model: hidden dimension 8192, #heads 64, #layers 80, vocab size 32k, context length 2048
- Much better performance compared to previous opensource models

$$\text{FFN}_{\text{SwiGLU}}(x, W, V, W_2) = (\text{Swish}_1(xW) \otimes xV)W_2$$

		BoolQ	PIQA	SIQA	HellaSwag	WinoGrande	ARC-e	ARC-c	OBQA
GPT-3	175B	60.5	81.0	-	78.9	70.2	68.8	51.4	57.6
Gopher	280B	79.3	81.8	50.6	79.2	70.1	-	-	-
Chinchilla	70B	83.7	81.8	51.3	80.8	74.9	-	-	-
PaLM	62B	84.8	80.5	-	79.7	77.0	75.2	52.5	50.4
PaLM-cont	62B	83.9	81.4	-	80.6	77.0	-	-	-
PaLM	540B	88.0	82.3	-	83.4	81.1	76.6	53.0	53.4
LLaMA	7B	76.5	79.8	48.9	76.1	70.1	72.8	47.6	57.2
	13B	78.1	80.1	50.4	79.2	73.0	74.8	52.7	56.4
	33B	83.1	82.3	50.4	82.8	76.0	80.0	57.8	58.6
	65B	85.3	82.8	52.3	84.2	77.0	78.9	56.0	60.2

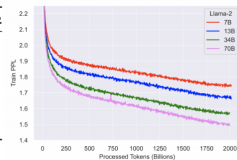
LLaMA: Open and Efficient Foundation Language Models [Touvron *et al.*, 2023]

Llama 2

Better performance from a larger training corpus

- Larger context length (2k -> 4k)
- More training tokens (1T/1.4T -> 2T, no sign of saturation yet)
- GQA for larger models (70B, 64 heads, 8 kv heads)
- Also include Llama-2-chat, an instruction-tuned version

Benchmark (shots)	GPT-3.5	GPT-4	PaLM	PaLM-2-L	LLAMA 2
MMLU (5-shot)	70.0	86.4	69.3	78.3	68.9
TriviaQA (1-shot)	–	–	81.4	86.1	85.0
Natural Questions (1-shot)	–	–	29.3	37.5	33.0
GSM8K (8-shot)	57.1	92.0	56.5	80.7	56.8
HumanEval (0-shot)	48.1	67.0	26.2	–	29.9
BIG-Bench Hard (3-shot)	–	–	52.3	65.7	51.2



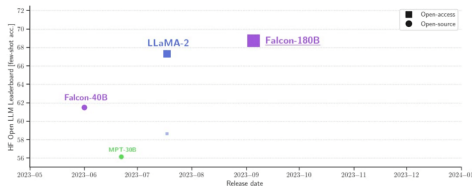
LLaMA: Open and Efficient Foundation Language Models [Touvron *et al.*, 2023]



Falcon

open-source LLM

- Size: 1.3B/7.5B/40B/**180B**
- **180B** model: hidden dimension 14848, #heads 232, #kv heads 8, #layers 80, vocab size 65k
- Comparable performance as PaLM-2 Large



LLaMA: Open and Efficient Foundation Language Models [Touvron *et al.*, 2023]

Multi-modal LLMs

Towards LLMs that can see

- GPT-4V(ision) can take visual inputs

Prompt:

How much did I pay for tax? Prompt Continued
Where is the information?



GPT-4V:

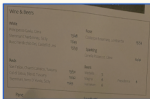
In the first receipt, you paid \$3.72 for tax. The information is located at the bottom of the receipt. In the second receipt, you paid \$42.23 for tax. The information is located near the bottom of the receipt, under the "Sales Tax" line. In the third receipt, you paid \$4.58 for tax. The information is located at the bottom of the receipt, under the "TAX" line. In total, you paid \$50.53 in tax across all three receipts.

Prompt:

How much should I pay for the beer on the table according to the price on the menu?



Here is a photo of the menu.



GPT-4V:

According to the menu, the price for a Magna beer is \$6. Since there are two Magna beers on the table, the total cost for the beers would be \$12.

Prompt:

Please read the text in this image and return the information in the following JSON format (note xxx is placeholder, if the information is not available in the image, put "N/A" instead)
{ "class": xxx, "DLN": xxx, "DOB": xxx, "Name": xxx, "Address": xxx, "EXP": xxx, "ISS": xxx, "SEX": xxx, "HGT": xxx, "WGT": xxx, "EYES": xxx, "HAIR": xxx, "DONOR": xxx }



GPT-4V:

{ "class": "D", "DLN": "D08954796", "DOB": "01/01/1974", "Name": "Jelani Sample", "Address": "123 Main St, Phoenix, AZ 85007", "EXP": "03/01/2024", "ISS": "03/01/2016", "SEX": "M", "HGT": "5-09", "WGT": "185 lbs", "EYES": "BRO", "HAIR": "N/A", "DONOR": "Y" }

The Dawn of LMMs: Preliminary Explorations with GPT-4V(ision) [Yang et al., 2013]

Multi-modal LLMs

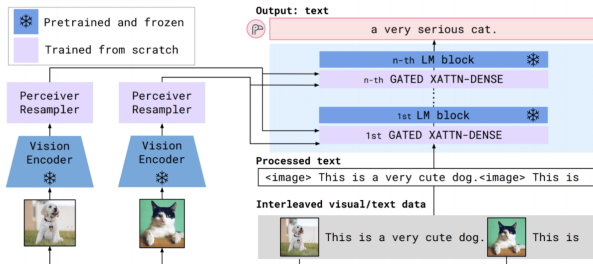
Two ways for visual language models

- Cross-attention from visual to LLM (Flamingo)
- Visual tokens as input (PaLM-E)

Flamingo

Cross-attention-based VLMs

- Pioneer work on enabling visual input support for LLMs
- LLM is frozen, added cross-attention layers to the intermediate layers of LLMs



Flamingo: a Visual Language Model for Few-Shot Learning [Alayrac et al., 2022]

Taking visual inputs (or other modalities) as tokens

- [illegible]

48/43